

Fichas de Pipeline

Uma ficha por pipeline em produção na Plataforma de Dados MinC: objetivo, fonte, destino, periodicidade, dependências e política de reprocessamento de cada uma das treze DAGs do Airflow.

Universidade de Brasília

Lab Livre · Faculdade do Gama, UnB

Ministério da Cultura

Gov Hub BR

Projeto de Pesquisa

Plataforma de Dados do Ministério da Cultura · Gov Hub BR

Meta 02 · Produto 5

Documentação dos pipelines de dados.

Meta 03 · Produto 3

Scripts e procedimentos de implantação.

Documento

Fichas de Pipeline



Índice

01	Objetivo e escopo	4
02	Panorama das DAGs	5
03	Fundo a fundo, dados cadastrais	6
04	A cadeia dos anexos	8
05	BB Ágil, SALIC e território	10
06	A DAG do dbt	12
07	Cobertura e lacunas	13





Este documento reúne a ficha técnica de cada pipeline em operação na Plataforma de Dados MinC. Os dados de cada ficha foram extraídos diretamente do código das DAGs, e não de uma descrição paralela que possa divergir do que executa.

13

DAGs no repositório

12

de ingestão

1

de transformação

4

sistemas de origem

O que cada ficha registra

Campo	De onde vem
Objetivo	Docstring do módulo ou comentários da DAG.
Fonte	Cliente de API ou extrator importado pela DAG.
Destino	Schema e tabela do <code>insert</code> , via as constantes de <code>plugins/schemas_minc.py</code> .
Periodicidade	Parâmetro <code>schedule</code> do decorador <code>@dag</code> .
Tarefas	Funções decoradas com <code>@task</code> , na ordem de declaração.
Reprocessamento	<code>retries</code> e <code>retry_delay</code> dos <code>default_args</code> .
Responsável	Campo <code>owner</code> dos <code>default_args</code> .
Etiquetas	Parâmetro <code>tags</code> do decorador <code>@dag</code> .



PERIODICIDADE AJUSTÁVEL SEM NOVO DEPLOY

Sete DAGs declaram `get_dynamic_schedule("")` em vez de um cron fixo. A função lê a Variable `dynamic_schedules` do Airflow e aceita preset, cron ou intervalo; quando não há entrada para aquela DAG, o padrão é `@daily`. Mudar a periodicidade é editar uma Variable, não publicar código.





DAG	Origem	Periodicidade	Resp.
<code>api_programas_dag</code>	TransfereGov	Dinâmica	C. Borges
<code>api_planos_acao_dag</code>	TransfereGov	Dinâmica	W. Souza
<code>api_plano_acao_meta_dag</code>	TransfereGov	Dinâmica	C. Borges
<code>api_plano_acao_dado_bancario_dag</code>	TransfereGov	Dinâmica	C. Borges
<code>api_movimentacoes_financeiras_dag</code>	TransfereGov	Dinâmica	C. Borges
<code>api_relatorios_gestao_dag</code>	TransfereGov	Manual	C. Borges
<code>api_anexos_relatorios_dag</code>	TransfereGov	Manual	C. Borges
<code>download_anexos_transferegov_dag</code>	TransfereGov	Manual	W. Souza
<code>extracao_anexos_dag</code>	Anexos XLSX	Manual	C. Borges
<code>extracao_bbagil_dag</code>	BB Ágil / BSC	Dinâmica	C. Borges
<code>salic_ingestion</code>	SALIC / SQL Server	Diária	W. Souza
<code>ingest_territorio_fcu_dag</code>	IBGE CD2022	Manual	Meta 3
<code>minc_cosmos_dag</code>	dbt (interno)	Diária, 01h	Meta 3

Periodicidade *Dinâmica* significa que o valor vem da Variable `dynamic_schedules`, com padrão `@daily`. *Manual* significa `schedule=None`: a DAG é disparada sob demanda.



A CADEIA DOS ANEXOS

Quatro DAGs formam uma cadeia sequencial, cada uma consumindo o que a anterior gravou: plano de ação, relatório de gestão, anexo e planilha. Todas as quatro são manuais, porque a etapa seguinte só faz sentido depois que a anterior concluiu.





Cinco DAGs extraem os dados cadastrais e financeiros do TransfereGov via `cliente_transferegov_fundo_a_fundo` e gravam no schema `transferegov`. O escopo de programas vem da Variable `transferegov_programas_ids`, com onze programas como padrão.

api_programas_dag

TransfereGov · API fundo a fundo

Objetivo	Carregar os programas do MinC monitorados pelas quatro políticas.
Destino	<code>transferegov.programa_minc</code>
Periodicidade	Dinâmica, padrão <code>@daily</code>
Tarefas	<code>fetch_programas</code> → <code>load_programas_to_postgres</code>
Reprocessa	3 tentativas, intervalo de 5 min
Etiquetas	minc · transferegov · programas · raw

api_planos_acao_dag

TransfereGov · API fundo a fundo

Objetivo	Carregar os planos de ação dos programas monitorados, com enriquecimento de território via <code>territorio_ibge</code> .
Destino	<code>transferegov.plano_acao_minc</code>
Periodicidade	Dinâmica, padrão <code>@daily</code>
Tarefas	<code>fetch_planos_acao</code> → <code>load_planos_to_postgres</code>
Reprocessa	3 tentativas, intervalo de 5 min
Etiquetas	minc · transferegov · planos_acao · raw

api_plano_acao_meta_dag

TransfereGov · API fundo a fundo

Objetivo	Carregar as metas de cada plano de ação, iterando sobre os planos já gravados.
Depende de	<code>transferegov.plano_acao_minc</code>
Destino	<code>transferegov.plano_acao_meta_minc</code>
Periodicidade	Dinâmica, padrão <code>@daily</code>
Tarefas	<code>fetch_metas</code> → <code>load_metas_to_postgres</code>
Reprocessa	3 tentativas, intervalo de 5 min

api_plano_acao_dado_bancario_dag

TransfereGov · API fundo a fundo

Objetivo	Carregar os dados bancários de cada plano de ação. É o insumo que identifica agência e conta para a extração do BB Ágil.
Depende de	<code>transferegov.plano_acao_minc</code>
Destino	<code>transferegov.plano_acao_dado_bancario_minc</code>
Periodicidade	Dinâmica, padrão <code>@daily</code>
Tarefas	<code>fetch_dados_bancarios</code> → <code>load_dados_bancarios_to_postgres</code>
Reprocessa	3 tentativas, intervalo de 5 min

api_movimentacoes_financeiras_dag

TransfereGov · API de gestão financeira

Objetivo	Extrair lançamentos e subtransações de gestão financeira, com pouso intermediário no MinIO antes da carga no Postgres.
Destino	<code>transferegov.raw_gestao_financeira_lancamentos</code> e <code>...subtransacoes</code>
Periodicidade	Dinâmica, padrão <code>@daily</code>
Tarefas	4, alternando extração para o MinIO e carga no Postgres
Reprocessa	3 tentativas, intervalo de 5 min





Quatro DAGs manuais formam a cadeia que vai do relatório de gestão até a planilha carregada no banco. Cada etapa consome o que a anterior gravou, e por isso nenhuma delas tem periodicidade automática.

api_relatorios_gestao_dag

TransfereGov · API fundo a fundo

Objetivo	Listar os relatórios de gestão de cada plano de ação.
Depende de	<code>transferegov.plano_acao_minc</code>
Destino	<code>transferegov.relatorios_gestao</code>
Periodicidade	Manual (<code>schedule=None</code>)
Reprocessa	3 tentativas, intervalo de 5 min

api_anexos_relatorios_dag

TransfereGov · API fundo a fundo

Objetivo	Listar os anexos de cada relatório de gestão, sem baixar o conteúdo.
Depende de	<code>transferegov.relatorios_gestao</code>
Destino	<code>transferegov.anexos_relatorios</code>
Periodicidade	Manual (<code>schedule=None</code>)
Reprocessa	2 tentativas, intervalo de 5 min

download_anexos_transferegov_dag

TransfereGov · API fundo a fundo

Objetivo	Baixar o conteúdo dos anexos ainda pendentes e gravar os bytes no MinIO.
Depende de	<code>transferegov.anexos_relatorios</code>
Destino	MinIO, bucket criado sob demanda
Periodicidade	Manual (<code>schedule=None</code>)
Reprocessa	2 tentativas, intervalo de 5 min



extracao_anexos_dag

Anexos XLSX · via `extracao_planilhas`

Objetivo	Abrir as planilhas anexadas, rotear cada aba para a tabela correspondente e carregar as linhas no schema de relatório de gestão.
Depende de	<code>transferegov.anexos_relatorios</code> e os arquivos no MinIO
Destino	As seis tabelas <code>relatorio_gestao.planilha_*</code>
Periodicidade	Manual (<code>schedule=None</code>)
Tarefas	<code>listar_anexos_pendentes</code> → <code>baixar_e_extrair</code> → <code>fechar_pipeline</code>
Reprocessa	2 tentativas, intervalo de 5 min
Idempotência	Chave natural <code>hash_registro</code> , derivada de identificador do anexo, índice da subtabela e linha de origem. Reprocessar o mesmo anexo atualiza as linhas em vez de duplicá-las.



A DAG MAIS COMPLEXA DO REPOSITÓRIO

Com 716 linhas, `extracao_anexos_dag` é a maior do projeto. Ela lida com planilhas de estrutura variável, em que a mesma informação aparece com nomes de coluna diferentes, e resolve o roteamento por prefixo do nome da aba. É também a única com teste automatizado:

`tests/test_conformidade_planilhas.py` cobre o roteamento das abas para as seis tabelas de destino.

**extracao_bbagil_dag**BB Gestão Ágil / SERPRO · via `<code>cliente_bsc</code>`

Objetivo	Extrair o extrato bancário e as subtransações das contas dos entes federados, identificadas a partir dos dados bancários do plano de ação.
Depende de	<code>transferegov.plano_acao_dado_bancario_minc</code> e <code>programa_minc</code>
Destino	<code>bbagil.extrato_bbagil</code> , <code>bbagil.subtransacao_bbagil</code> e duas tabelas de controle de retomada
Periodicidade	Dinâmica, padrão <code>@daily</code>
Tarefas	<code>carregar_contas_bancarias</code> → <code>extrair_extrato_bbagil</code> → <code>extrair_subtransacoes_bbagil</code>
Reprocessa	100 tentativas, intervalo de 10 min
Situação	Bloqueada: a autenticação SCA falha por variável <code>SCA_TOKEN_URL</code> não preenchida.

**POR QUE 100 TENTATIVAS NÃO É EXAGERO**

O serviço do BB Ágil aplica bloqueio temporário após uso sustentado, entre 20 e 40 minutos de chamadas contínuas, independentemente do ritmo. A DAG persiste no Postgres a cada 2.000 itens processados, de modo que cada nova tentativa refaz apenas o que não foi gravado desde o último lote. Com esse ponto de retomada, insistir por muitas horas sem supervisão é barato; desistir depois de poucas tentativas jogaria fora horas de execução.



salic_ingestion

SALIC · SQL Server, via `sql_server_extractor`

Objetivo	Extrair e carregar em bruto as tabelas dos servidores SALIC na camada bronze do Postgres. Nenhuma transformação em Python: todos os valores pousam como texto, e a tipagem fica com o dbt.
Configuração	Variable <code>salic_data</code> em JSON, com <code>conn_id</code> , banco, schema, tabelas e tamanho de lote por fonte. Tabelas vazias significa extrair o schema inteiro.
Conexões	<code>mssql_salic_<servidor></code> e <code>postgres_default</code>
Destino	Schema <code>bronze</code> ; registro de execução em <code>control.salic_ingestion_log</code>
Periodicidade	<code>@daily</code>
Tarefas	<code>load_config</code> → <code>expand_targets</code> → <code>ensure_schemas</code> → <code>extract_and_load</code>
Reprocessa	3 tentativas, intervalo de 5 min; lote padrão de 50.000 linhas

ingest_territorio_fcu_dag

IBGE · Censo 2022, arquivo CSV

Objetivo	Carregar o cruzamento entre setor censitário, concentração urbana, município e UF. É a base da quarta cota, a territorial.
Entrada	<code>data/external/territorio/fcu_setores_2022.csv</code>
Destino	<code>transferegov.territorio_fcu_setores</code>
Periodicidade	Manual. O dado é estável entre censos.
Reprocessa	1 tentativa, intervalo de 2 min
Teste	<code>tests/test_territorio_ibge.py</code> , com 8 casos





Uma única DAG executa todo o projeto dbt. Ela é construída pelo Cosmos, que lê o projeto e gera automaticamente uma tarefa do Airflow por modelo e por teste, em vez de encapsular tudo num comando opaco.

minc_cosmos_dag

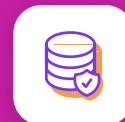
Projeto dbt `<code>minc</code>, via astronomer-cosmos`

Objetivo	Transformar bronze em silver e gold nos dois domínios, e executar as 923 verificações de qualidade.
Entrada	Todas as tabelas de pouso gravadas pelas DAGs de ingestão
Saída	36 modelos nos schemas <code>agentes</code> , <code>minc_cotas</code> e <code>metadata</code>
Periodicidade	Diária, 01h00 (<code>0 1 * * *</code>)
Perfil	<code>dbt/minc/profiles.yml</code> , destino <code>prod</code>
Reprocessa	2 tentativas
Histórico	Desligado (<code>catchup=False</code>)



GRANULARIDADE POR MODELO, NÃO POR COMANDO

Como o Cosmos expande o projeto em tarefas individuais, uma falha aparece na interface do Airflow apontando exatamente o modelo ou o teste que quebrou. Reprocessar significa reexecutar aquela tarefa, e não o projeto inteiro.



Uma fonte prevista sem pipeline

O plano de documentação prevê ficha de pipeline para TransfereGov, SALIC e Mapas Culturais. As duas primeiras estão documentadas neste documento. **A terceira não existe como pipeline neste repositório.**

O Mapas Culturais aparece apenas como fonte declarada no projeto dbt, com 29 tabelas em `dados_mapa_cultura`. A própria descrição da fonte registra que a estrutura foi inferida do esquema público do projeto e não foi verificada contra a base real. Não há DAG de ingestão, nenhum modelo consome essas tabelas, e portanto não há ficha a emitir.



DUAS FONTES DOCUMENTADAS QUE O PLANO NÃO PREVIA

Em contrapartida, duas fontes com pipeline em operação não constavam do escopo previsto e estão documentadas aqui: o BB Ágil, com a DAG mais tolerante a falha do repositório, e o cruzamento territorial do IBGE, que sustenta a quarta cota da Meta 3.

Três lacunas de documentação nas próprias DAGs

1. **Onze das treze DAGs não têm docstring.** Só `salic_ingestion` e `ingest_territorio_fcu_dag` descrevem o próprio objetivo no módulo. Nas demais, o objetivo registrado nesta ficha foi reconstruído a partir do nome das tarefas, dos comentários internos e do destino da carga.
2. **O campo `owner` mistura pessoa e frente.** Onze DAGs nomeiam uma pessoa; `ingest_territorio_fcu_dag` traz `Meta 3 - cotas`. Padronizar esse campo é pré-requisito para a matriz de papéis e responsabilidades prevista na Meta 02.
3. **Não há alerta configurado.** Nenhuma DAG declara notificação em caso de falha. A detecção depende de alguém abrir a interface do Airflow.